



Stellenbosch

UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

Statistical Learning Theory: Assignment 1

Lise Visagie

June 26, 2026

Stellenbosch University

Question 1

Supervised learning in a broad context

Introduction

According to [Hastie et al. \(2009\)](#), there are three types of statistical learning problems, namely supervised, unsupervised and semi-supervised learning. Supervised learning algorithms aim to predict one or more quantities, referred to as the response value (vector) $\mathbf{Y} \in \mathbb{R}^q$, using variables $\mathbf{X} \in \mathbb{R}^p$, also called features. An example of such a learning problem is predicting the probability of a patient having a heart attack based on features such as their heart rate and blood pressure. In contrast, unsupervised learning algorithms only observe features and aim to learn how data is organised or clustered. Clustering methods can, for example, be applied to clinical data to identify groups of patients with different survival rates. As the name, semi-supervised, suggests, this learning problem combines supervised and unsupervised learning, with features present, but only some observations have corresponding responses. Supervised learning will be the focus for the remainder of this chapter.

The probabilistic approach to supervised learning

As stated previously, the objective of supervised statistical learning algorithms is to predict some unknown response $\mathbf{Y}$ from the corresponding features $\mathbf{X}$ with some general real-valued function $f : \mathcal{X} \rightarrow \mathbb{R}^p$ referred to as the candidate model. This model is estimated by fitting it to observed data, $D = \{(\mathbf{x}_i, \mathbf{y}_i) : i = 1, \dots, N\}$ where N denotes the number of observations, $\mathbf{x}_i \in \mathcal{X} \in \mathbb{R}^p$ and $\mathbf{y}_i \in \mathbb{R}^q$. [Hastie et al. \(2009\)](#) claims this objective is achieved by assuming that $(\mathbf{X}, \mathbf{Y})$ are random vectors with a probability density function $\tilde{\mathbb{P}}_{\mathbf{X}, \mathbf{Y}} : \mathcal{X} \times \mathcal{Y} \rightarrow [0, \infty)$ such that the observations in D are identically, independently distributed according to $\tilde{\mathbb{P}}$. It follows that supervised learning methods, therefore, try to fit a model, $\hat{f}(\mathbf{x})$, to the observed data, D , to predict $\mathbf{Y}$ with $\hat{f}(\mathbf{X})$.

Depending on the response data type, different supervised learning problems arise. When $\mathbf{Y}$ is numeric, predictions are made with regression and more specifically, if $q > 1$, the problem becomes multiple output regression. When $\mathbf{Y}$ is categorical, the aim is to solve a classification problem. As with regression, classification can be branched out further into binary, multi-class and multi-label classification. Binary classification problems predict a binary categorical variable, $Y \in \{0, 1\}$. When Y needs to be classified into one of $k > 2$ classes, the problem is multi-class classification, but when the response can be classified into more than one class and a vector of categorical variables is therefore predicted, the problem is multi-label classification ([Hastie et al., 2009](#)).

To illustrate the difference between these supervised learning problems, consider, for example, the medical domain in which various predictions can be made. If the probability of a single patient getting infected with a disease should be predicted, the predicted value will be a continuous number between zero and one and therefore, this is a regression problem. When one wants to predict whether a patient is infected or not, the predicted outcomes can be encoded as zero if the patient is not infected and one otherwise. This is therefore a binary classification problem. When a patient is suffering from one of ten possible diseases, and the aim is to predict which disease the patient is suffering from, this becomes a multi-class classification problem since the response is categorical with more than two possible outcomes.

The role of loss functions

Hastie *et al.* (2009) claims that supervised learning problems can be solved with discriminative or generative modelling. The generative approach aims to estimate the distribution of $\mathbf{Y}|\mathbf{X}$ by first estimating $\hat{\mathbb{P}}_{\mathbf{X},\mathbf{Y}}$. Finally, the estimated distribution of $\mathbf{Y}|\mathbf{X}$ is then used for prediction. In contrast, discriminative modelling focuses on estimating $\hat{\mathbb{P}}_{\mathbf{X}}$ nonparametrically in order to extract information from the distribution $\mathbf{Y}|\mathbf{X}$ to ultimately make predictions. The discriminative approach, therefore, estimates the distribution of $\mathbf{Y}|\mathbf{X}$ directly by training a model, $\hat{f}$, using the structure of $\mathbf{X}$.

There are various models that can be used to predict $\mathbf{Y}$. According to Hastie *et al.* (2009), discriminative modelling defines the optimal model as $\hat{f} = \operatorname{argmin}_f[\hat{\mathcal{R}}(f)]$. Therefore, the optimal model, also known in this setting as the population minimiser, minimises the population risk, $\hat{\mathcal{R}}(f) = E_{\mathbf{X},\mathbf{Y}}[\mathcal{L}(\mathbf{Y}, f(\mathbf{X}))]$. In this formulation $\mathcal{L}(\mathbf{Y}, f(\mathbf{X}))$ is some loss function $\mathcal{L} : \mathcal{Y} \times \mathbb{R}^r \rightarrow [0, \infty)$ that quantifies the loss occurred when $f(\mathbf{X})$ is used to predict $\mathbf{Y}$. Equivalently, the model that minimises the loss function will be the best model.

Just as different models can be found to fit data, different loss functions can be employed to find the best model. Intuitively, different loss functions will yield different optimal models. To show this, consider two well-known loss functions, the squared-error loss, $\mathcal{L}(Y, f(\mathbf{X})) = (Y - f(\mathbf{X}))^2$, and absolute loss, $\mathcal{L}(Y, f(\mathbf{X})) = |Y - f(\mathbf{X})|$. The optimal model for $\mathbf{X} = \mathbf{x}$ using the squared-error loss is derived as (Hastie *et al.*, 2009):

$$\begin{aligned}\hat{f}(\mathbf{x}) &= \operatorname{argmin}_f[E((Y - f(\mathbf{X}))^2|\mathbf{X} = \mathbf{x})] \\ &= \operatorname{argmin}_f[E(Y^2|\mathbf{X} = \mathbf{x}) - 2f(\mathbf{x})E(Y|\mathbf{X} = \mathbf{x}) + f(\mathbf{x})^2] \\ \therefore \text{The expression will be minimized when } f(\mathbf{x})E(Y|\mathbf{X} = \mathbf{x}) &= f(\mathbf{x})^2 \\ \therefore \hat{f}(\mathbf{x}) &= E(Y|\mathbf{X} = \mathbf{x})\end{aligned}$$

It follows that a similar procedure can be followed for the absolute loss, and although the calculations are not as simple, it can be derived that the population minimiser at $\mathbf{X} = \mathbf{x}$ is equal to the median of $\mathbf{Y}|\mathbf{X} = \mathbf{x}$. The squared error loss is simpler to work with, but it is less robust than the absolute loss. Loss functions should therefore be carefully chosen after considering the nature of the supervised learning problem.

Another common loss function, specifically used in classification, is the 0-1 loss,

$$\mathcal{L}_{0-1}(Y, f(\mathbf{X})) = \begin{cases} 0 & \text{if } Y = f(\mathbf{X}), \\ 1 & \text{if } Y \neq f(\mathbf{X}). \end{cases}$$

The derivation for finding the corresponding optimal model follows as:

$$\begin{aligned}\hat{f}(\mathbf{x}) &= \operatorname{argmin}_f[E(\mathbb{I}(Y \neq f(\mathbf{x}))|\mathbf{X} = \mathbf{x})] \\ &= \operatorname{argmin}_f[p(\mathbf{x})\mathbb{I}(1 \neq f(\mathbf{x})) + (1 - p(\mathbf{x}))\mathbb{I}(0 \neq f(\mathbf{x}))] \\ &= \operatorname{argmin}_f[p(\mathbf{x})\mathbb{I}(0 = f(\mathbf{x})) + (1 - p(\mathbf{x}))(1 - \mathbb{I}(0 = f(\mathbf{x})))] \\ &= \operatorname{argmin}_f \begin{cases} p(\mathbf{x}) & \text{if } f(\mathbf{x}) = 0, \\ 1 - p(\mathbf{x}) & \text{if } f(\mathbf{x}) = 1. \end{cases} \\ \therefore \hat{f}(\mathbf{x}) &= \begin{cases} 0 & \text{if } p(\mathbf{x}) < 0.5, \\ 1 & \text{if } p(\mathbf{x}) \geq 0.5. \end{cases}\end{aligned}$$

Since this loss function is used in classification problems, the optimal model is not a specific quantity, as with the squared error loss and absolute error loss, but rather a decision boundary. There are, therefore, multiple combinations of parameters which are within this boundary. To show how this would look visually, this 0-1 loss function was applied to a logistic regression problem, which required patients to be classified as heart attack risk patients ($Y=1$) or not ($Y=0$) based on their age (X_1) and cholesterol levels (X_2). The first step in this application is to simulate a sample of size $n = 200$ from the true data generating mechanism, $Y \sim \text{Ber}(p)$ with $p = [1 + e^{-\mathbf{X}\beta}]^{-1}$ where $\beta' = [\beta_0, \beta_1, \beta_2] = [-2, 0.02, 0.01]$, $\mathbf{X}' = [X_1, X_2]$, $X_1 \sim N(50, 10)$ and $X_2 \sim N(200, 30)$.

```
import numpy as np
import matplotlib.pyplot as plt

Beta = [-2, 0.02, 0.01]

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def generate_data(n):
    age = np.random.normal(loc=50, scale=10, size=n)
    cholesterol = np.random.normal(loc=200, scale=30, size=n)
    X = np.column_stack((age, cholesterol))
    X_design = np.c_[np.ones(X.shape[0]), X]
    z = X_design @ Beta
    p = sigmoid(z)
    y = np.random.binomial(1, p)

    return X, y

np.random.seed(123)
X, y = generate_data(1000)
```

Next, a heatmap was constructed to visualise $E(\mathcal{L}_{0-1}(Y, f(\mathbf{X})))$ for varying values of β_1 and β_2 while keeping β_0 fixed.

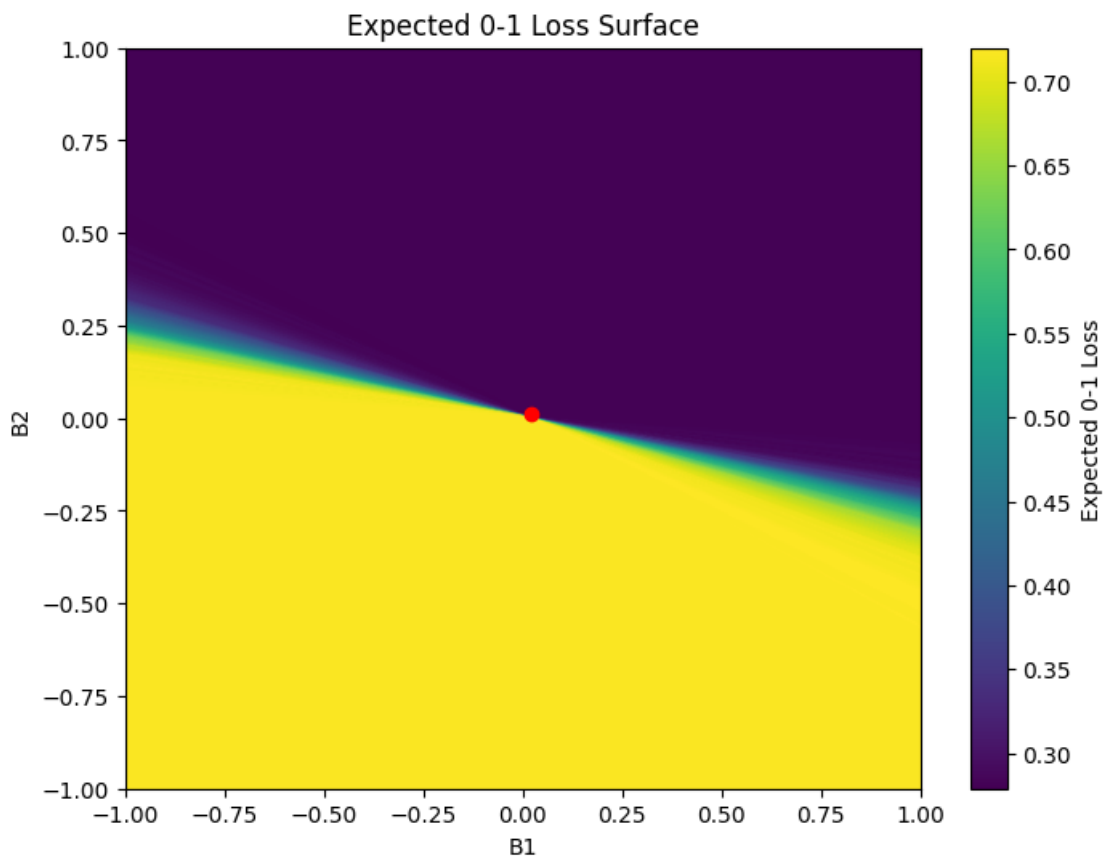
```
import numpy as np
import matplotlib.pyplot as plt

b1 = np.linspace(-1, 1, 500)
b2 = np.linspace(-1, 1, 500)
B1, B2 = np.meshgrid(b1, b2)
X_design = np.c_[np.ones(X.shape[0]), X]

loss = np.zeros_like(B1)
for i in range(B1.shape[0]):
    for j in range(B1.shape[1]):
        beta = np.array([-2, B1[i, j], B2[i, j]])
        p_model = sigmoid(X_design @ beta)
        yhat = (p_model >= 0.5).astype(int)
```

```
loss[i, j] = np.mean(yhat != y)
```

```
plt.figure(figsize=(8,6))
plt.imshow(loss, origin='lower', extent=[b1.min(), b1.max(), b2.min(), b2.
↪max()], aspect='auto')
plt.plot(0.02,0.01,"ro")
plt.colorbar(label="Expected 0-1 Loss")
plt.xlabel("B1")
plt.ylabel("B2")
plt.title("Expected 0-1 Loss Surface")
plt.show()
```



As expected, the heatmap does not show $E(\mathcal{L}_{0-1}(Y, f(\mathbf{X})))$ to have a minimum at a certain point, but rather indicates a decision boundary at the true parameter values. This aligns with what was derived analytically.

Empirical risk and regularisation

In the previous subsection, it was discussed how to find the optimal model for a supervised learning problem. When we only consider models in the model space $\mathcal{F}$, $\hat{f}$ can be approximated with

$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} [\hat{\mathcal{R}}(f)]$. To find a $\hat{f}$, however, the data from the whole population or the true data-generating mechanism is required. Since this is almost never available in practice, the population minimiser can be approximated with an optimal model, $\hat{f}$ that minimises the empirical risk, $\hat{\mathcal{R}}(f, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$. This is equivalent to the optimisation problem

$$\hat{f}(\cdot, \mathcal{D}) = \operatorname{argmin}_{f \in \mathcal{F}} [\hat{\mathcal{R}}(f, \mathcal{D})]. \quad (1.1)$$

The empirical risk is a sufficient estimator for the population risk since $\hat{\mathcal{R}}(f, \mathcal{D}) \xrightarrow{P} \mathcal{R}(f)$ as $|\mathcal{D}| \rightarrow \infty$. To ensure that $\hat{f}(\cdot, \mathcal{D})$ is also a sufficient estimator for $\hat{f}$ the following must hold:

$$\sup_{f \in \mathcal{F}} [|\hat{\mathcal{R}}(f, \mathcal{D}) - \mathcal{R}(f)|] \xrightarrow{P} 0 \text{ as } |\mathcal{D}| \rightarrow \infty.$$

This condition can only be achieved when suitable constraints are imposed on $\mathcal{F}$, such as regularisation. According to Hastie et al. (2009), regularisation is a learning algorithm that partially minimises the empirical risk. Regularisation introduces bias into the model by reducing model complexity. This will enhance model performance on unseen data, or, differently stated, it will improve model generalisation.

Types of regularisation techniques include variable selection and functional regularisation. Variable selection reduces model complexity by excluding certain features from the feature set that is used to train the model, to potentially remove noise and improve model generalisation. This approach is referred to as hard regularisation. In contrast, functional regularisation reduces model complexity by introducing a penalty function $\mathcal{P} : \mathcal{F} \rightarrow [0, \infty)$ which changes the model space to $\mathcal{F}(t) = \{f \in \mathcal{F} : \mathcal{P} \leq t\}$. With functional regularisation, the optimisation problem (1.1) therefore becomes:

$$\hat{f}(\cdot, \mathcal{D}, \lambda) = \operatorname{argmin}_{f \in \mathcal{F}} [\hat{\mathcal{R}}(f) + \lambda \mathcal{P}(f)].$$

Continuing the logistic regression example from before, functional regularisation, specifically ridge regularisation, can be applied to potentially improve generalisation performance. For ridge regularisation, the penalty function becomes $\mathcal{P}(f) = \|E_{\mathbf{X}}[\frac{\partial f}{\partial \mathbf{X}}]\|_2^2$ and when applied to logistic regression, this reduces to $\beta' \beta$. The following implementation performed logistic regression with and without regularisation, while setting the tuning parameter $\lambda = 1$.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

def train_models():
    model_unreg = LogisticRegression( C=np.inf, fit_intercept=True,
    ↪ random_state=0, max_iter=1000)
    model_wridge = LogisticRegression( C=1, fit_intercept=True, random_state=0,
    ↪ max_iter=1000)
    model_sridge = LogisticRegression( C=0.005, fit_intercept=True,
    ↪ random_state=0, max_iter=1000)

    return model_unreg, model_wridge, model_sridge

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```

model_unreg, model_wridge, model_sridge = train_models()
model_unreg.fit(X_scaled, y)
model_wridge.fit(X_scaled, y)
model_sridge.fit(X_scaled, y)

print("Unregularised Logistic Regression")
print("Intercept   : ",model_unreg.intercept_)
print("Coefficients: ",model_unreg.coef_[0])

print("\nRidge Logistic Regression with C = 1")
print("Intercept   : ",model_wridge.intercept_)
print("Coefficients: ",model_wridge.coef_[0])

print("\nRidge Logistic Regression with C = 0.005")
print("Intercept   : ",model_sridge.intercept_)
print("Coefficients: ",model_sridge.coef_[0])

```

```

Unregularised Logistic Regression
Intercept   :  [0.97870688]
Coefficients:  [0.326      0.33730111]

```

```

Ridge Logistic Regression with C = 1
Intercept   :  [0.97822383]
Coefficients:  [0.32414595 0.33538198]

```

```

Ridge Logistic Regression with C = 0.005
Intercept   :  [0.94557328]
Coefficients:  [0.15615171 0.16186327]

```

The tuning parameter C plays a significant role in the amount by which a model is regularised. When C is large, model complexity is penalised less than when C is chosen small. From the results, it is clear that as regularisation increased, estimates moved closer to the true parameter values. It is naive, however, to make assumptions about the generalisation performance of the different models by just comparing the parameter estimates, which motivates the reason for computing generalisation errors.

Importance of measuring the generalisation performance

When a model is too complex and fits the observed data too well, overfitting occurs. It captured random fluctuations along with the true underlying pattern in the data, which causes poor generalisation. In contrast, underfitting occurs when the model is so simple that it cannot learn the underlying pattern at all, which will also lead to poor generalisation. It is therefore important that the generalisation performance is measured to evaluate a fitted model and to ensure that neither overfitting nor underfitting occurs.

There are different measurements for generalisation performance. The generalisation error,

$$Err(\hat{f}, \mathcal{D}) = E_{\mathbf{X}, \mathbf{Y}}[\mathcal{L}(\mathbf{Y}, \hat{f}(\mathbf{X}, \mathcal{D}))],$$

measures the error of a specific fitted model on unseen data. Since the true data-generating mechanism is unknown, the error must be estimated. The training error, $e\bar{r}r(\hat{f}, \mathcal{D}) = \hat{\mathcal{R}}(\hat{f}(\cdot, \mathcal{D}), \mathcal{D})$ is considered a downward-biased estimator for the generalisation error since it fits and assesses the model with the same data [Hastie et al. \(2009\)](#).

A better estimator for the generalisation error is the test error, $\hat{E}rr(\hat{f}, \mathcal{D}, \mathcal{T}_e) = \hat{\mathcal{R}}(\hat{f}(\cdot, \mathcal{D}), \mathcal{T}_e)$. This estimate is obtained by first dividing the available data into a set $\mathcal{D}$, for model estimation and a set $\mathcal{T}_e$, for performance assessment. One of the test error's attractive properties is that it becomes consistent as the $|\mathcal{T}_e|$ grows if the data-generating mechanism satisfies the i.i.d. assumption.

In contrast, the expected generalisation error, $Err(\hat{f}) = E_{\mathcal{D}}[Err(\hat{f}, \mathcal{D})]$ measures the error of a learning algorithm. According to [Hastie et al. \(2009\)](#), this error can be estimated with simulation. First, generate a large number, say B, data sets from the data-generating mechanism and split each set into $(\mathcal{D}^{(b)}, \mathcal{T}_e^{(b)})$ for $b=1, \dots, B$. It follows that the expected generalisation error can be estimated with $\hat{E}rr(\hat{f}, B) = \frac{1}{B} \sum_{b=1}^B \hat{E}rr(\hat{f}, \mathcal{D}^{(b)}, \mathcal{T}_e^{(b)})$.

To explore how regularisation influences the generalisation performance of a model, the training, test and estimated expected generalisation error were computed for the logistic models previously trained.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import zero_one_loss

def fit_and_evaluate(model, X_train, y_train, X_test, y_test):
    model.fit(X_train, y_train)
    train_err = zero_one_loss(y_train, model.predict(X_train))
    test_err = zero_one_loss(y_test, model.predict(X_test))

    return train_err, test_err

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
    ↪random_state=0)

# Obtain test and training errors
model_unreg, model_wridge, model_sridge = train_models()
train_err_unreg, test_err_unreg = fit_and_evaluate(model_unreg, X_train,
    ↪y_train, X_test, y_test)
train_err_wridge, test_err_wridge = fit_and_evaluate(model_wridge, X_train,
    ↪y_train, X_test, y_test)
train_err_sridge, test_err_sridge = fit_and_evaluate(model_sridge, X_train,
    ↪y_train, X_test, y_test)

# Obtain expected test errors
def estimate_expected_error(X, y, C, B=100, test_size=0.3):

    err = []
    n = len(y)
```

```

np.random.seed(123)
for b in range(B):
    boot = np.random.choice(n, size=n, replace=True)
    X_b = X[boot]
    y_b = y[boot]

    X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(X_b, y_b,
↳test_size=test_size)
    model = LogisticRegression( C=C,fit_intercept=True, random_state=0,
↳max_iter=1000)

    _, test_err = fit_and_evaluate(model, X_train_b, y_train_b, X_test_b,
↳y_test_b)
    err.append(test_err)

return np.mean(err)

exp_err_unreg = estimate_expected_error(X_scaled, y, C=np.inf,B=100)
exp_err_wridge = estimate_expected_error(X_scaled, y, C=1,B=100)
exp_err_sridge = estimate_expected_error(X_scaled, y, C=0.005,B=100)

print("=== Training and test errors ===")
print("\nUnregularised Logistic Regression:")
print(f"Training error : {train_err_unreg:.6f}")
print(f"Test error      : {test_err_unreg:.6f}")

print("\nRidge Logistic Regression (L2) with C = 1:")
print(f"Training error : {train_err_wridge:.6f}")
print(f"Test error      : {test_err_wridge:.6f}")

print("\nRidge Logistic Regression (L2) with C = 0.005:")
print(f"Training error : {train_err_sridge:.6f}")
print(f"Test error      : {test_err_sridge:.6f}")

print("\n=== Estimated Expected Generalisation Error (simulation) ===")
print(f"Unregularised          : {exp_err_unreg:.6f}")
print(f"Lightly Regularised (C=1) : {exp_err_wridge:.6f}")
print(f"Heavily Regularised (C=0.005): {exp_err_sridge:.6f}")

```

=== Training and test errors ===

Unregularised Logistic Regression:

Training error : 0.275714

Test error : 0.303333

Ridge Logistic Regression (L2) with C = 1:

Training error : 0.274286

Test error : 0.303333

Ridge Logistic Regression (L2) with C = 0.005:

Training error : 0.270000

Test error : 0.310000

=== Estimated Expected Generalisation Error (simulation) ===

Unregularised : 0.278000

Lightly Regularised (C=1) : 0.277667

Heavily Regularised (C=0.005): 0.277667

The training error, which is a downward-biased estimator for the generalisation error, decreases as regularisation increases. The test errors, which provide a much more accurate reflection of the generalisation performance of a fitted model, indicated that the regularised model with hyperparameter C=0.005 generalised the worst. This implies the model complexity was penalised too much and therefore underfits the data. In contrast, the estimated expected generalisation error of the unregularized model was higher than both the regularised models, indicating the regularised learning algorithm generalises better.

From this example, it is evident that regularisation should be conducted carefully, as too little or too much regularisation could lead to underfitting or overfitting. To find the optimal balance, the bias-variance trade-off should be examined.

Bias-variance trade-off and the curse of high dimensionality

Bias refers to how close an estimate is to the true value. When model complexity is increased, there is a greater risk for overfitting and the bias of the model is decreased. Variance refers to how stable an estimate is. When model complexity is increased and the model starts to overfit the data, the predictions for unseen data will be less stable and the variance of the model will increase [Hastie et al. \(2009\)](#).

It follows that when bias is decreased by increasing model complexity, the variance will increase. This implies a negative relationship between bias and variance, which is known as the bias-variance trade-off. When a model has a large bias but low variance, underfitting occurs, causing poor generalisation. Conversely, when a model has a low bias but large variance, overfitting occurs, which also leads to poor generalisation. The aim, therefore, in statistical learning problems, is to find a model that minimises the generalisation error, which can only be achieved by minimising both bias and variance.

This trade-off is found in both regression and classification. [Hastie et al. \(2009\)](#) states that the bias and variance can be minimised in regression settings by minimising the mean squared error (MSE). This error can be expressed as $MSE(\hat{f}) = bias(\hat{f})^2 + var(\hat{f})$, which is a useful decomposition of the bias-variance trade-off. In this expression $bias(\hat{f})^2 = E_{\mathbf{X}} \left[(\hat{f}(\mathbf{X}) - E_{\mathcal{D}}[\hat{f}(\mathbf{X}, \mathcal{D})])^2 \right]$ and $var(\hat{f}) = E_{\mathbf{X}} \left[Var_{\mathcal{D}}[\hat{f}(\mathbf{X}, \mathcal{D})] \right]$.

In classification, decomposing the bias-variance trade-off is not as simple. Various metrics have been developed over the years, but none as simple and elegant as the MSE. [Domingos \(2000\)](#) discovered that for certain loss functions such as the squared error and 0-1 loss, the generalisation error can

express the bias-variance decomposition as $Err(\hat{f}, \mathcal{D}) = E_{\mathbf{X}, \mathbf{Y}}[\mathcal{L}(\mathbf{Y}, \hat{f}(\mathbf{X}, \mathcal{D}))] = c_1 Noise + Bias + c_2 Variance$ where c_1 and c_2 are coefficients depending on the loss function.

It therefore follows that the hyperparameter C in our ongoing example should be chosen such that it minimises both the bias and variance. To illustrate how the choice of C influences the bias and variance, a plot thereof was constructed for the ridge logistic regression model evaluated at varying values of C . From the plot, it is clear that where the bias and variance intersect, the optimal tradeoff of combined bias and variance was minimised. At this point, C ranged between 0.03 and 0.06, implying that when $C \in [0.03, 0.06]$, generalisation performance was enhanced.

```
def bias_variance_logistic(C_values, n=200, B=100, N=1000):
    bias_list = []
    var_list = []

    np.random.seed(123)
    X, _ = generate_data(N)
    X_design = np.c_[np.ones(X.shape[0]), X]
    p_true = sigmoid(X_design @ Beta)

    for C in C_values:
        preds_all = []

        for b in range(B):
            X_train, y_train = generate_data(n)
            scaler = StandardScaler()
            Xt_scaled = scaler.fit_transform(X_train)

            model = LogisticRegression( C=C, fit_intercept=True, random_state=0,
↪max_iter=1000)
            model.fit(Xt_scaled, y_train)

            X_scaled = scaler.transform(X)
            p_hat = model.predict_proba(X_scaled)[: , 1]
            preds_all.append(p_hat)

        preds_all = np.array(preds_all)
        p_mean = preds_all.mean(axis=0)

        bias = np.mean((p_mean - p_true)**2)
        var = np.mean(np.var(preds_all, axis=0))

        bias_list.append(bias)
        var_list.append(var)

    return np.array(bias_list), np.array(var_list)

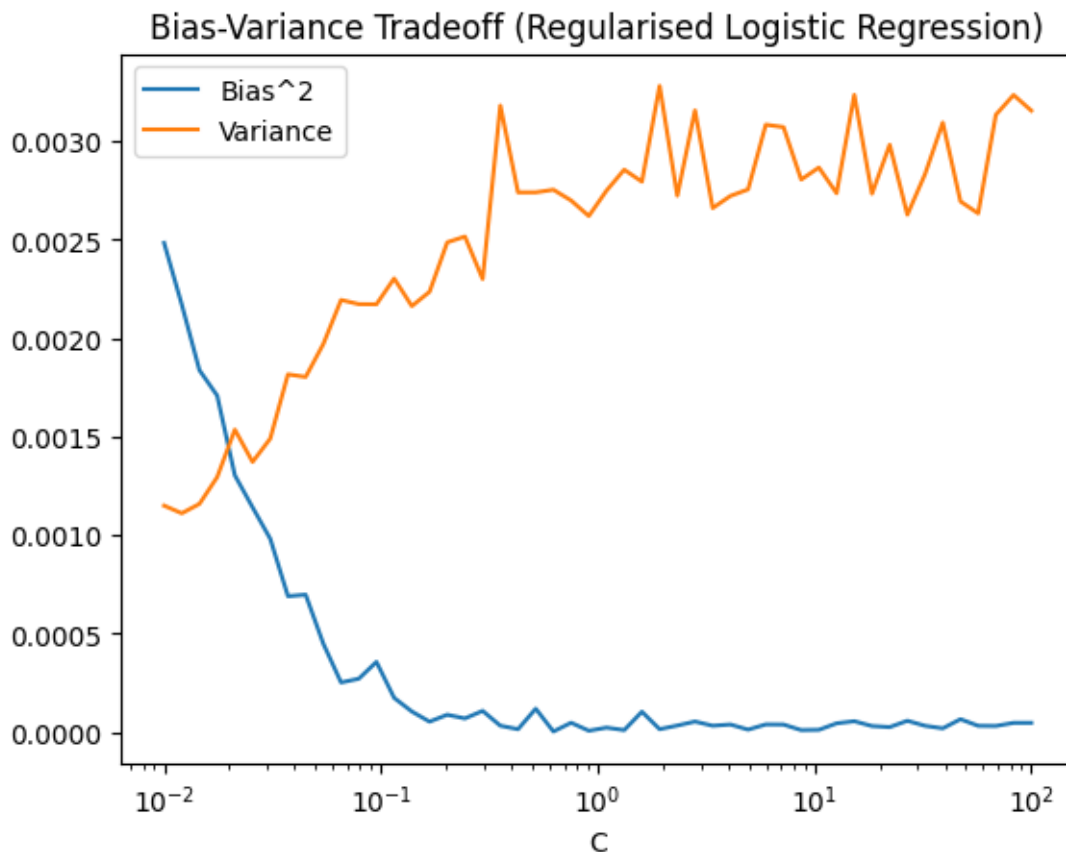
C_values = np.logspace(-2, 2, 50)
```

```

bias, variance = bias_variance_logistic( C_values, n=200, B=100, N=1000)

plt.figure()
plt.plot(C_values, bias, label="Bias^2")
plt.plot(C_values, variance, label="Variance")
plt.xscale('log')
plt.xlabel("C")
plt.title("Bias-Variance Tradeoff (Regularised Logistic Regression)")
plt.legend()
plt.show()

```



Model assessment and selection techniques

Thus far, different attributes of an optimal model have been discussed. The model that minimises the empirical risk or alternatively, the applicable loss function, should be selected for prediction. This model should have the lowest bias and variance and should generalise well. There are two techniques, the validation-set approach and cross-validation, which can be used to select and assess such an optimal model.

The validation-set approach is usually applied when enough data is available. In line with [Hastie](#)

et al. (2009), this technique splits all the data into two sets, $\mathcal{D}^*$ and the test set $\mathcal{T}_e$. This is followed by further splitting $\mathcal{D}^*$ into the training set $\mathcal{T}_r$ and validation set $\mathcal{V}$. The algorithm continues by training all possible models on $\mathcal{T}_r$. It then selects the model which minimised the empirical risk when applying models to $\mathcal{V}$. Finally, the selected model is assessed on $\mathcal{T}_e$ with metrics such as the generalisation error.

Cross-validation is similar to the validation-set approach, but performs more data partitioning and is often used when there is not a lot of data available. To perform model selection as well as model assessment, all the data, as with the validation-set approach, should be split into two sets, $\mathcal{D}^*$ and the test set $\mathcal{T}_e$. The algorithm then divides $\mathcal{D}^*$ into K mutually exclusive and exhaustive groups $\mathcal{V}^{(k)} : k = 1, \dots, K$ called folds. When $K=1$, this technique is called leave-one-out cross-validation (LOOCV). Next, each possible model is trained K times, each time, k , on $\mathcal{D}^*$ excluding fold k . It then selects the model, $\hat{f}$, which minimises the cross-validation error, $cver(\hat{f}, \mathcal{D}, K) = \frac{1}{K} \sum_{k=1}^K \hat{\mathcal{R}}(\hat{f}(\cdot, \mathcal{D}^* \setminus \mathcal{V}^{(k)}), \mathcal{V}^{(k)})$, where $\mathcal{D}^* \setminus \mathcal{V}^{(k)}$ refers to $\mathcal{D}^*$ excluding fold k (Hastie *et al.*, 2009). Exactly like the validation set approach, the selected model is assessed on $\mathcal{T}_e$ with metrics such as the generalisation error.

Selecting the best model in the regularised logistic regression application is equivalent to finding the optimal C value. Using 10-fold cross-validation, models with $C \in [0.04, 0.07]$, similar to before, minimise the cross-validation error and can be regarded as sensible model selections. Assessing these models with the test errors indicates, however, that models with C values slightly smaller ($C \in [0.02, 0.05]$) yields better generalisation performance for the current dataset.

```
from sklearn.model_selection import KFold

np.random.seed(123)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.5,
    random_state=0)

def kfold_cv_error(X, y, C, K=5):
    kf = KFold(n_splits=K, shuffle=True, random_state=0)

    errors = []

    for train_idx, val_idx in kf.split(X):
        X_tr, X_val = X[train_idx], X[val_idx]
        y_tr, y_val = y[train_idx], y[val_idx]

        model = LogisticRegression( C=C, fit_intercept=True, random_state=0,
    max_iter=1000)
        model.fit(X_tr, y_tr)
        errors.append(zero_one_loss(y_val, model.predict(X_val)))

    return np.mean(errors)

C_values = np.logspace(-2, 2, 100)
cv_errors = []
```

```

test_errors = []
exp_errors = []

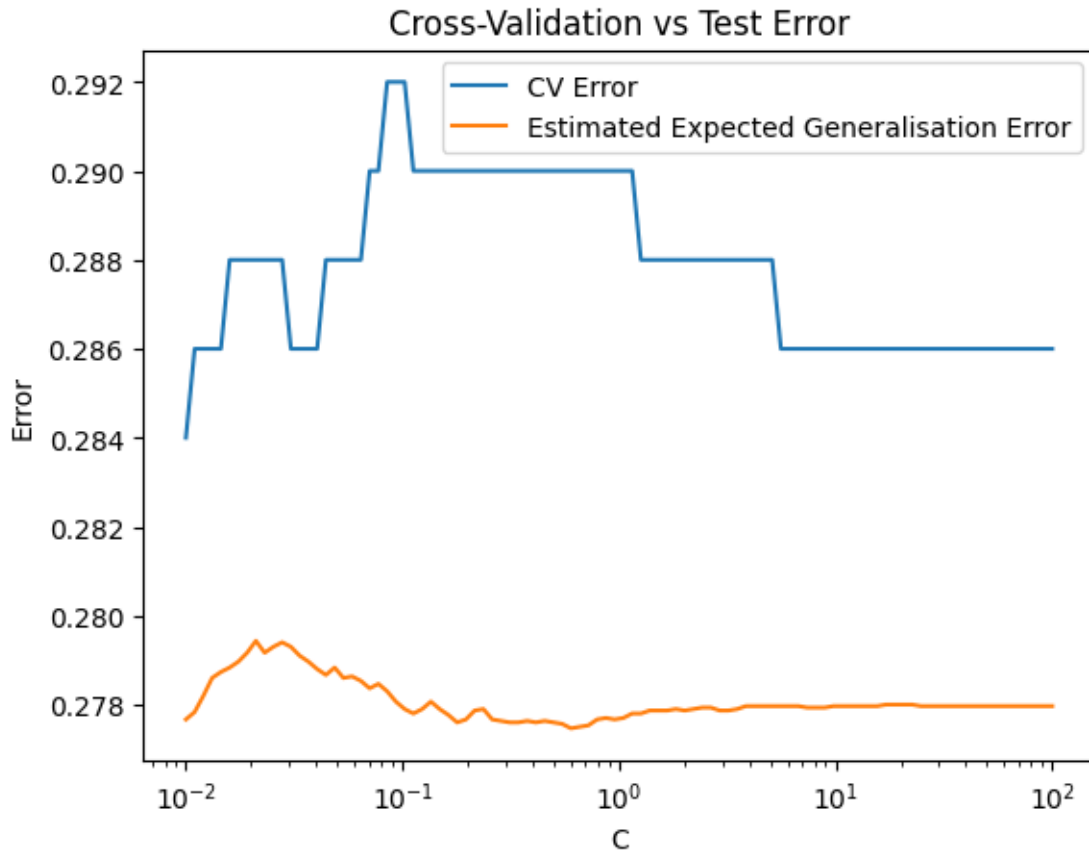
for C in C_values:
    cv_errors.append(kfold_cv_error(X_train, y_train, C, K=10))

    model = LogisticRegression( C=C, fit_intercept=True, random_state=0,
    ↪max_iter=1000)
    model.fit(X_train, y_train)
    test_errors.append(zero_one_loss(y_test, model.predict(X_test)))

    exp_errors.append(estimate_expected_error(X_scaled, y, C=C,B=100))

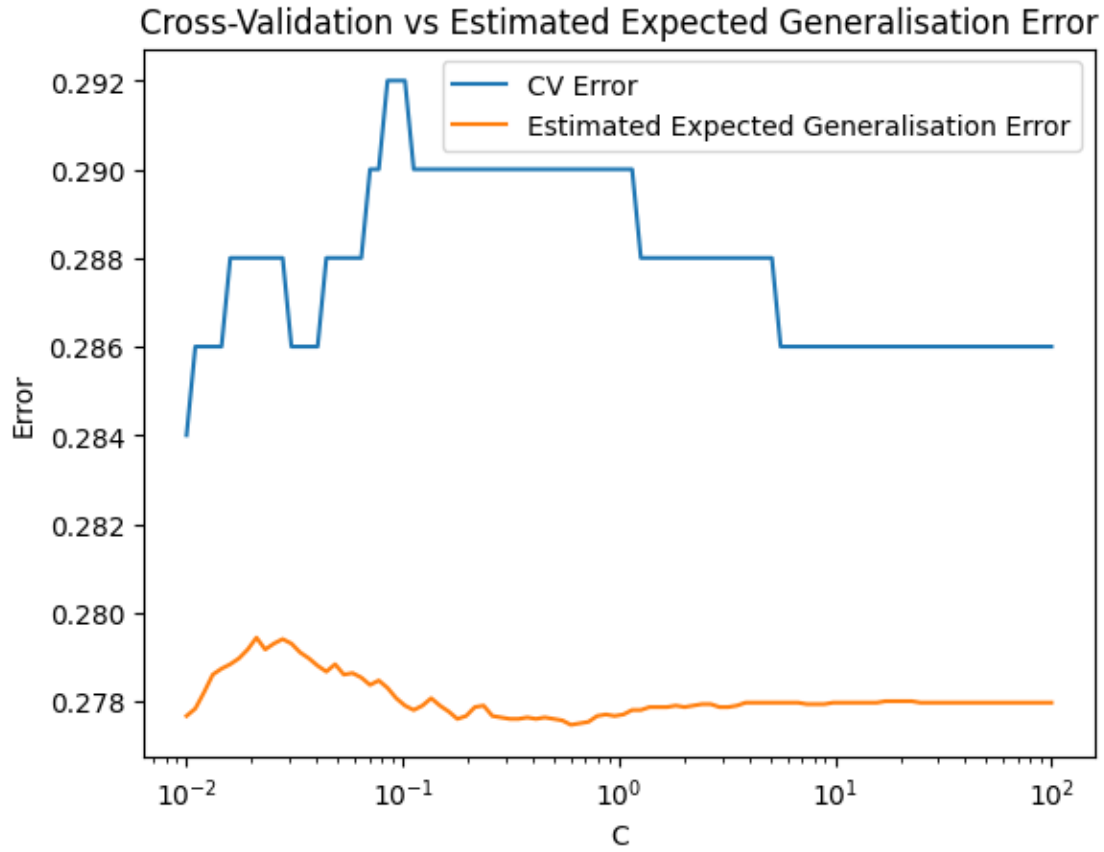
plt.figure()
plt.plot(C_values, cv_errors, label="CV Error")
plt.plot(C_values, test_errors, label="Test Error")
plt.xscale('log')
plt.xlabel("C")
plt.ylabel("Error")
plt.title("Cross-Validation vs Test Error")
plt.legend()
plt.show()

```



Interestingly, the cross-validation error can also serve as an estimator for the expected generalisation error. This is because models are trained on K different datasets and evaluated on unseen data. Again, the bias-variance trade-off can be observed when choosing the value of K . When K is large, the bias is small and the variance is large, since models will be trained and evaluated on more diverse datasets. This concept is visualised with the following plot.

```
plt.figure()
plt.plot(C_values, cv_errors, label="CV Error")
plt.plot(C_values, exp_errors, label="Estimated Expected Generalisation Error")
plt.xscale('log')
plt.xlabel("C")
plt.ylabel("Error")
plt.title("Cross-Validation vs Estimated Expected Generalisation Error")
plt.legend()
plt.show()
```



Conclusion

Supervised learning provides a structured framework for predicting responses from observed features, with the choice of loss functions, regularisation and model selection techniques playing critical roles in achieving optimal performance. Through the logistic regression example, it was shown that while regularisation can improve generalisation, too much penalisation leads to underfitting, and too little risks overfitting. Careful evaluation using training, test, and expected generalisation errors, alongside bias-variance analysis and cross-validation, ensures models achieve the right balance between bias and variance. Ultimately, the effectiveness of supervised learning lies in selecting models that minimise generalisation error while maintaining robustness and interpretability.

Question 2

The analysis of a practical data set

Introduction

Silver is not only a financial asset or a critical industrial metal, but it also serves as an economic indicator. Accurate silver price predictions have therefore become essential for investors, manufacturers and the government, enabling them to manage risk effectively, seize opportunities and implement proactive policy decisions.

Predicting the return of silver is usually of greater importance than predicting the silver price since the return measures investment performance and purchasing power trends rather than just a single, isolated data point. The financial industry regards the autoregressive integrated moving average (ARIMA) time series model as the benchmark model for predicting these return values, but since the residuals of return values are often white noise, predictions can become meaningless. Instead, powerful results are obtained by predicting the volatility with a generalised autoregressive conditional heteroskedasticity (GARCH) time series model.

As the statistical field evolved, ensemble models such as extreme gradient boosting (XGBoost) started to gain popularity for their impressive accuracy and efficiency. These models are more complex and effectively exploit information gained from additional features instead of just previously observed values. This assignment will examine the relationship between model complexity and generalisation performance, while comparing the predictive performance of the benchmark time series model, ARIMA-GARCH, with XGBoost when predicting log return values and their volatility, respectively.

Objectives and research question

This assignment will investigate and compare the benchmark ARIMA-GARCH model and the ensemble model, XGBoost, when predicting both the log return values of silver in South Africa and their volatility. Log returns will be used since it provides mathematical convenience. The ARIMA and GARCH components will be trained on the log return series, while XGBoost models will be fitted using lagged log returns and volatility measures. In addition, XGBoost will be extended with relevant external features to enhance both log return and volatility prediction. Models of varying complexity will be considered, with model selection performed with 10-fold expanding window cross-validation. The aim is to understand how model complexity influences predictive accuracy and which model framework is most appropriate for the respective target predictions. The objectives that will guide this study are:

1. To examine the effect of model complexity on predictive performance.
2. To examine the influence of external features on predictive performance.
3. To compare the predictive performance of time series models and an ensemble model.

The core research question that will guide this study is thus: Which statistical model and at which level of complexity is predictive performance optimised for each target?

Limitations

This assignment had several limitations. Firstly, the fundamental differences in the architectures of time series models and XGBoost made it challenging to ensure fully meaningful comparisons. To maintain comparability, certain conditions were imposed, which may have prevented some models from demonstrating their full strengths. Furthermore, while hyperparameter tuning was performed for XGBoost models, it was not feasible to carry out an exhaustive search due to computational constraints. Thus, the chosen values may not have been fully optimal.

Data description

Financial data¹ was retrieved using Yahoo Finance’s API and the Federal Reserve Economic Data (FRED) API for the period 01 January 2025 to 11 June 2026. Following data extraction and feature engineering, the final dataset consisted of 239 observations and 37 variables. These variables included log return values of silver, which were used to fit and test the ARIMA and GARCH models. Tables 1 and 2 indicate the variables used to fit and train the XGBoost models for predicting log returns and their volatility, respectively.

Table 1: Features for predicting log returns.

Feature	Description
return_lag_(i)	Lagged log returns of silver, $i = 1, \dots, 5$
momentum_(i)	Difference in South African silver prices over $i = 7, 14$ trading days
MA_ratio_short	Ratio of 7-day to 30-day moving averages of silver (trend-strength indicator)
sp500_returns	Daily log returns of the S&P 500 Index
gold_returns	Daily log returns of gold prices
brent_returns	Daily log returns of Brent crude oil prices
dxy_returns	Daily log returns of the U.S. Dollar Index (broad USD strength)
fx_returns	Daily log returns of the USD/ZAR exchange rate (bilateral FX risk)
vix_changes	Daily percentage change in the CBOE Volatility Index (market risk sentiment)
gold_silver_ratio	Ratio of gold to silver prices (relative valuation)
gold_fx_interaction	Interaction term between gold returns and USD/ZAR returns
US_Interest_Rate	Effective Federal Funds Rate (short-term U.S. policy rate)
US_10Y_Yield	10-Year U.S. Treasury yield (benchmark long-term interest rate)
target_return	Next-period log return of silver (forecast target)

¹Data and code to retrieve it are accessible via: <https://github.com/LiseVisagie/SLT-Ass1.git>

Table 2: Features for predicting volatility.

Feature	Description
volatility_(i)	Rolling standard deviation of silver log returns over $i = 7, 14, 21, 63$ trading days
volatility_ratio	Ratio of 7-day to 14-day volatility, measuring relative short-term risk intensity
gold_vol_21	21-day rolling standard deviation of gold log returns
sp500_vol_21	21-day rolling standard deviation of S&P 500 log returns
fx_vol_21	21-day rolling standard deviation of USD/ZAR log returns
brent_vol_21	21-day rolling standard deviation of Brent crude oil log returns
vix_changes	Daily percentage change in the CBOE Volatility Index (risk sentiment shifts)
VIX	Level of the CBOE Volatility Index (expected 30-day equity market volatility)
US_10Y_Yield	10-Year U.S. Treasury yield (benchmark long-term interest rate)
target_vol	Next-period volatility proxy (5-day rolling standard deviation of silver log returns)

The relationships between variables in Table 1 are illustrated in Figure 1. The heatmap indicates relatively weak relationships between variables, except between `sp500_returns` and `vix_changes`. Since most variables are weakly correlated to the target variable, it was expected that they would have little influence on predicting log returns.

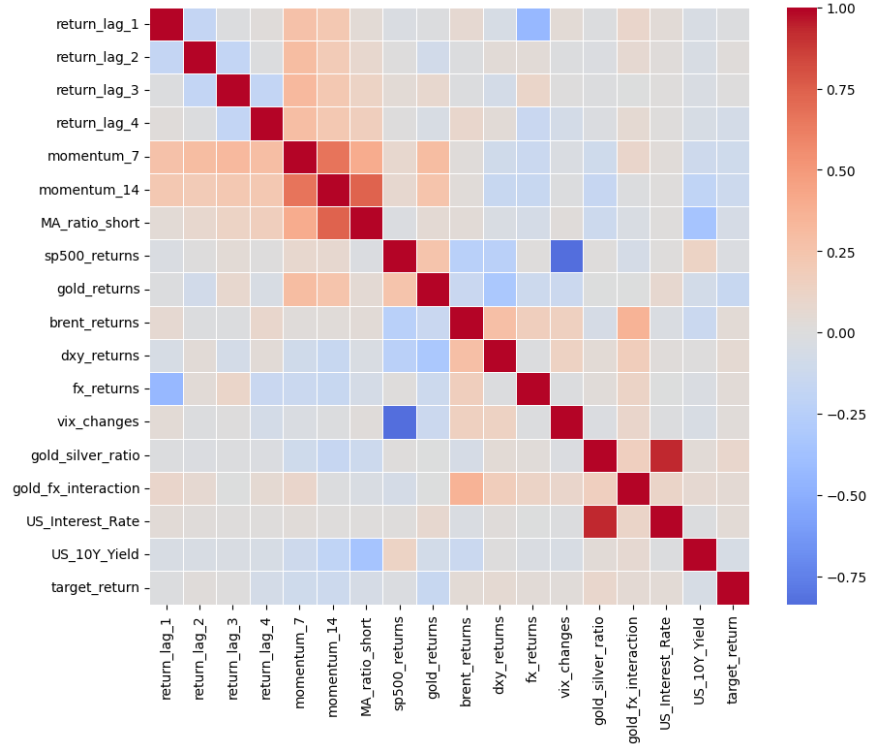


Figure 1: The correlation heatmap of features for predicting log returns.

Similarly, the relationships between variables in Table 2 are illustrated in Figure 2. The heatmap indicates relatively strong relationships between variables, contrasting with Figure 1. It was anticipated that these variables would have a greater influence on predicting volatility than the variables in Table 1 on predicting log returns.

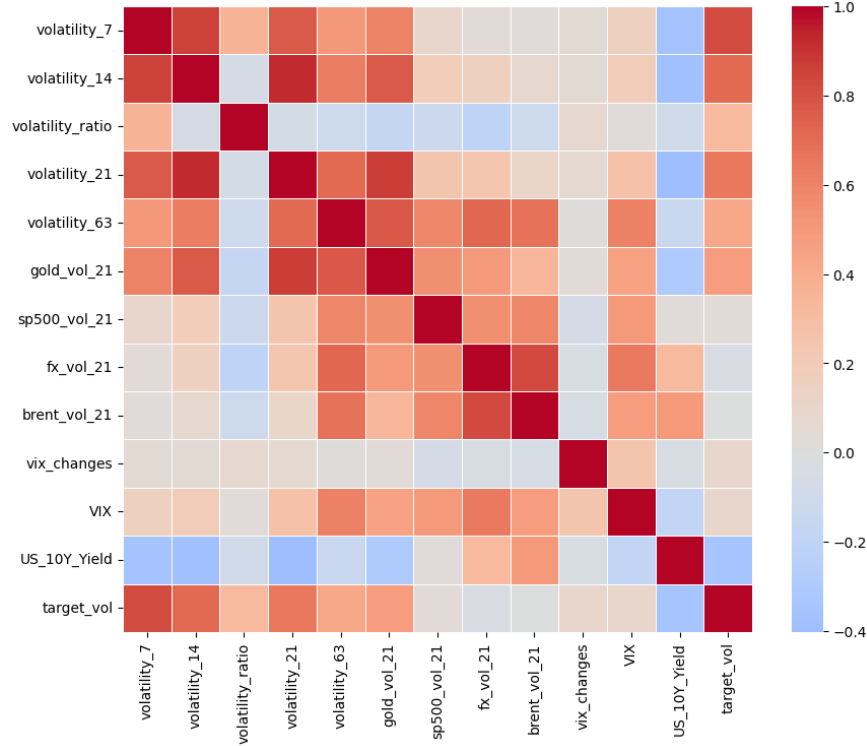


Figure 2: The correlation heatmap of features for predicting volatility of log returns.

Methodology

The assignment predicted log returns of silver and their volatility with an ARMA-GARCH model and XGBoost. The following section provides a summary of the theoretical background of these models as well as the model selection method, expanding window cross-validation.

ARIMA-GARCH

Any heteroscedastic time series can be expressed as $r_t = \mu_t + a_t$ at time point t , where μ_t is the mean equation and $a_t = \sigma_t \epsilon_t$ the sequence of residuals thereof. Further, σ_t^2 is known as the volatility equation and ϵ_t is a sequence of independent and identically distributed random variables that usually follow the standard normal or a standardised Student-t distribution. Since ϵ_t is random and therefore unpredictable, r_t is predicted with the mean equation, while the volatility equation indicates the certainty around the prediction. The ARIMA-GARCH model predicts the conditional mean with an ARIMA(p,d,q) model and the conditional variance with a GARCH(m,s) model (Viljoen, 2025a).

Viljoen (2025b) defines the ARIMA(p,d,q) model as:

$$\mu_t = \Phi_p(B)^{-1}(1 - B)^{-d}\Theta_q(B)\varepsilon_t,$$

where ε_t denotes a series of random errors and B denotes the backward shift operator. This operator shifts time back by 1 unit to form a new series such that $BX_t = X_{t-1}$ and $(1 - B)X_t = X_t - X_{t-1}$ where X_t could be μ_t or ε_t . To fit an ARIMA model, the timeseries u_t should be stationary and therefore have a constant mean and variance. This can be achieved by differencing the series with

$(1 - B)^d \mu_t$, where the value d denotes the number of times μ_t should be differenced until it is stationary.

The autoregressive operator, $\Phi_p(B) = (1 - \phi_1 B - \phi_2 B^2 - \dots - \phi_p B^p)$ and the moving average operator, $\Theta_q(B) = (1 - \theta_1 B - \theta_2 B^2 - \dots - \theta_q B^q)$ are polynomials with order p and q , respectively. The larger p and q , the more complex the model becomes. These values are often determined by examining the patterns formed by the autocorrelations and partial autocorrelations of the stationary time series. The parameters $\phi_1, \phi_2, \dots, \phi_p, \theta_1, \theta_2, \dots, \theta_q$ are known as regression weights and require estimation (Viljoen, 2025b).

The adequacy of this model is determined by testing whether the residuals of the model are from a white noise process. Therefore, if the sequence of residuals is uncorrelated and from a fixed distribution with a constant mean, usually zero, then the model is adequate and can be used for forecasting.

Viljoen (2025a) defines the GARCH(m, s) model as:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^m \alpha_i a_{t-i}^2 + \sum_{j=1}^s \beta_j \sigma_{t-j}^2,$$

where a_t denotes the residual series of the mean equation, which should be white noise. The residuals obtained from the fitted mean equation are used as input for this model. The parameters of the model follow the following constraints: $\alpha_0 > 0$, $\alpha_i \geq 0$ for $i = 1, \dots, m$, $\beta_j \geq 0$ for $j = 1, \dots, s$ and $\sum_{i=1}^{max(m,s)} (\alpha_i + \beta_j) < 1$. The reaction of conditional volatility is measured by α_i , while the persistence in conditional volatility is measured by β_j . Conditional maximum likelihood is often applied to estimate these parameters. As with the ARIMA model, the larger m and s , the more complex the model becomes.

Similar to ARIMA, the adequacy of GARCH models should be determined. This is achieved by testing whether the standardised residuals and standardised squared residuals of the model are from a white noise process. Residuals are standardised by dividing it with the conditional variance (Viljoen, 2025a).

XGBoost

XGBoost is a boosting algorithm that aims to improve the bias of weak base learners. Decision trees will be considered as base learners for this assignment. Gradient boosting improves bias by sequentially fitting trees to the residuals of previously trained trees and then adding them to obtain the final model for prediction and inference. These residuals are calculated with the negative gradient of some loss function.

According to Hastie *et al.* (2009), XGBoost improves gradient boosting by using the derivative as well as the second derivative to fit decision trees. This algorithm, therefore, exploits second-order information to gain greater accuracy with each decision tree being fit. The optimal tree in each iteration of the algorithm is obtained by solving:

$$\operatorname{argmin}_{\Theta} \{Q(\Theta)\} \approx \operatorname{argmin}_{\Theta} \left\{ \mathcal{L}(\mathbf{y}, \hat{\mathbf{f}}) + \frac{1}{2} \sum_{i=1}^n (h_{ii} \cdot \mathcal{T}(\mathbf{x}_i, \Theta) - 2r_i \cdot \mathcal{T}(\mathbf{x}_i, \Theta)) \right\}. \quad (2.1)$$

In equation (2.1), $\mathbf{y}$ refers to the vector of responses, while $\mathbf{x}_i$ refers to the features of observation i . $\mathcal{L}$ denotes the loss function, which is usually the squared error loss, $\hat{\mathbf{f}}$ denotes the current model and

$\mathcal{T}(\cdot, \Theta)$ denotes the decision tree with parameters Θ . Further, r_i and h_{ii} refer to the i^{th} element of the gradient and Hessian of $\mathcal{L}(\mathbf{y}, \hat{\mathbf{f}})$ respectively. It follows that the constants in each terminal region, $\mathcal{R}$ of the optimal decision tree is estimated by $\hat{c}(\mathcal{R}, \lambda) = \frac{\sum_{\mathbf{x}_i \in \mathcal{R}} r_i}{\lambda + \sum_{\mathbf{x}_i \in \mathcal{R}} h_{ii}}$ where λ is a hyperparameter parameter that acts as a L2 regulariser (Hastie *et al.*, 2009).

Before adding the new tree to the ensemble, it is pruned by moving up from the terminal nodes and removing splits until a split with a positive gain is encountered. Given that the hyperparameter, γ , determines how aggressively trees are pruned, the gain of splitting a terminal region $\mathcal{R}$ into $\mathcal{R}_{j,s}^-(\mathcal{R})$ and $\mathcal{R}_{j,s}^+(\mathcal{R})$ is defined by

$$\text{gain}(j, s, \mathcal{R}, \lambda, \gamma) = \frac{1}{2} \left[\frac{(\sum_{\mathbf{x}_i \in \mathcal{R}_{j,s}^-(\mathcal{R})} r_i)^2}{\lambda + \sum_{\mathbf{x}_i \in \mathcal{R}_{j,s}^-(\mathcal{R})} h_{ii}} + \frac{(\sum_{\mathbf{x}_i \in \mathcal{R}_{j,s}^+(\mathcal{R})} r_i)^2}{\lambda + \sum_{\mathbf{x}_i \in \mathcal{R}_{j,s}^+(\mathcal{R})} h_{ii}} \right] - \frac{1}{2} \frac{(\sum_{\mathbf{x}_i \in \mathcal{R}} r_i)^2}{\lambda + \sum_{\mathbf{x}_i \in \mathcal{R}} h_{ii}} - \gamma.$$

In line with Hastie *et al.* (2009), this model contains multiple hyperparameters besides λ and γ . The maximum tree depth can be specified with larger values corresponding to a more complex model with a greater risk of overfitting. The tree structure is also influenced by the minimum child weight. If this weight is relatively large, splits on nodes where the sum of the second derivative values is less than the weight will be eliminated. Additionally to *lambda*, which acts as an L2 regulariser, the L1 penalty can be introduced with α . Furthermore, the number of trees included in the ensemble and the amount each contributed to the final model, known as the learning rate, can affect the generalisation performance of the model. To reduce the risk of overfitting, XGBoost also implements subsampling. It can be specified as either the fraction of rows or features sampled per tree or the fraction of features sampled per split level or per node split.

Hyperparameters not only influence model complexity individually, but their interaction as well. Therefore, to simplify the examination of model complexity, only the maximum tree depth, minimum child weight, γ and λ were tuned.

Expanding window cross-validation

This technique is similar to normal k-fold cross-validation, but it is more suitable for time series data. It divides the data into $k+1$ equally sized sequential data blocks, where the first block is regarded as the initial training set. After training the model on the initial training set and validating it on the next data block, these blocks are combined to form the next training set. Models are always validated on the consecutive data block after the training set. The process of model training and validation continues until all the data is combined.

The cross-validation error is calculated in the same way as before, with the optimal model selected as the model which minimises this error. For this assignment, the cross-validation error will be calculated with the mean absolute error, since it is more robust and less sensitive to outliers.

Results

The models were trained and selected² with an 80% training set and assessed with a 20% test set. Since the data are from a time series, a 10-fold expanding window cross-validation was applied. First, the optimal ARIMA model was found and tested, after which the model's residuals were used

²Code accessible via: <https://github.com/LiseVisagie/SLT-Ass1.git>

to train, select and test the appropriate GARCH model. ARIMA(p,d,q) models for p,q=0,1 and GARCH(m,s) models for m,s =1,...,4 were considered for model selection.

XGBoost models were trained and tested for log returns and volatility prediction, respectively. For each target, the models were trained with and without external features. Models with a maximum tree depth that range between 1 and 5 and minimum child weight, γ and λ values within the set [0,0.5,1,1.5] were considered during model selection. As previously mentioned, all other XGBoost hyperparameters were kept constant. The learning rate was set to 0.01, while α was set to zero. All models contained 200 trees and no subsampling was performed.

Log return prediction

It was determined that the log return time series is stationary and does not require any differencing to fit an ARIMA(p,d,q) model and therefore d was kept constant at zero. The ARIMA(0,0,0) model that includes a constant mean produced the smallest cross-validation error of 0.03002 and achieved a test error of 0.02530 (Table 3).

Table 3: Cross-validation and test errors of the optimal models used to predict log returns.

Model	Cross-validation error	Test error
ARIMA	0.03002	0.02530
XGBoost	0.02996	0.02530
XGBoost with external features	0.02996	0.02530

The performance of XGBoost models fitted on the different feature subsets remained constant, suggesting that external features did not enhance log return prediction. The optimal models achieved a cross-validation error of 0.3002 and a test error of 0.02530 (Table 3). In both cases, hyperparameter tuning had little to no effect on the predictive performance. When external features were included, the cross-validation error remained constant across varying levels of model complexity, but when these features were excluded, the cross-validation error was minimised by enforcing $\gamma \geq 0.5$. Therefore, when incorporating only lagged log returns, simpler XGBoost models enhanced the generalisation performance.

The XGBoost models outperformed ARIMA during training, but they ultimately achieved the same test error. This implies that the greater complexity associated with ensemble methods did not translate into improved predictive accuracy for log returns. Even when external features were included, XGBoost failed to surpass ARIMA in identifying significant patterns within log return values. This is because the residuals behave as white noise, implying the absence of further predictable structure or complex non-linear relationships that XGBoost could exploit for enhanced predictions.

Volatility prediction

The GARCH(m,s) models were trained on the residuals of the ARIMA(0,0,0) model. The minimum cross-validation error (0.035) was attained with GARCH(3,2). A more complex model that delivered a lower test error of 0.028 (Table 4) was therefore selected.

Table 4: Cross-validation and test errors of the optimal models used to predict volatility of log returns.

Model	Cross-validation error	Test error
GARCH	0.03544	0.02842
XGBoost	0.01276	0.00735
XGBoost with external features	0.01387	0.00735

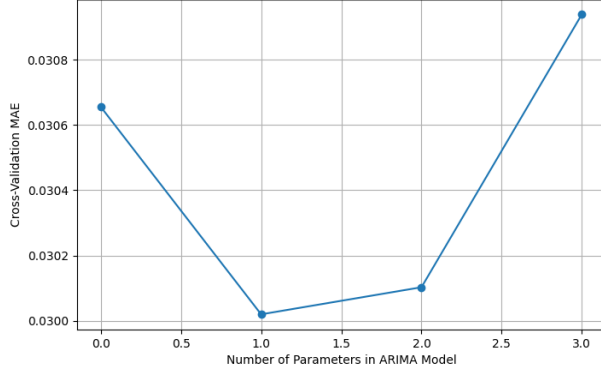
The cross-validation error (0.01387) obtained by the optimal XGBoost model fitted on the external features was higher than the error (0.01276) obtained by the optimal XGBoost model fitted only on lagged volatility observations (Table 3). These models performed equally well during testing, achieving an impressive test error of 0.00735. (Table 3). This suggests that including external features did not significantly affect the predictive performance of the models.

The same set of XGBoost hyperparameters was identified during model selection for both feature subsets. The maximum tree depth was one, and the minimum child weight was 1.5, while γ and λ were chosen as zero. Although the optimal minimum child weight, γ and λ values were relatively small, which promotes model complexity, the risk of overfitting was controlled with a low tree depth.

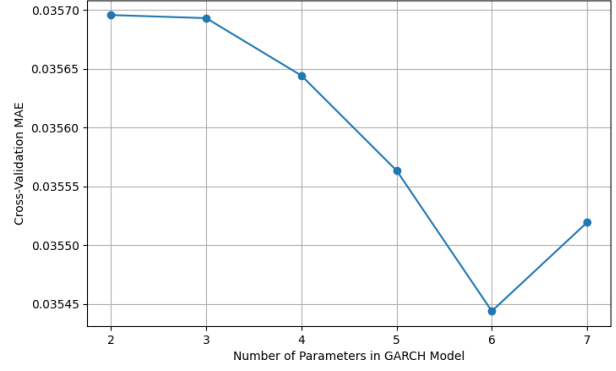
XGBoost consistently outperformed GARCH during both training and testing, regardless of whether external features were incorporated. When predicting volatility, the greater architectural complexity of XGBoost enabled it to capture more nuanced patterns in the data, which enhanced predictive performance.

Model suitability

Figure 3 illustrates that the ARMA model achieved optimal results with fewer parameters than the GARCH model. Distinct patterns also emerged across the XGBoost models. As shown in Figure 4, the cross-validation error for log return prediction was barely influenced by hyperparameter tuning, with optimality achieved when complexity was restricted with $\gamma \geq 0.5$. In contrast, volatility prediction exhibited increasing error as the values of hyperparameters increased. While deeper trees introduced greater complexity, larger values of the child weight, γ and λ encouraged simplicity. Thus, volatility prediction required more complex model structures to adequately capture the data dynamics than log return prediction.



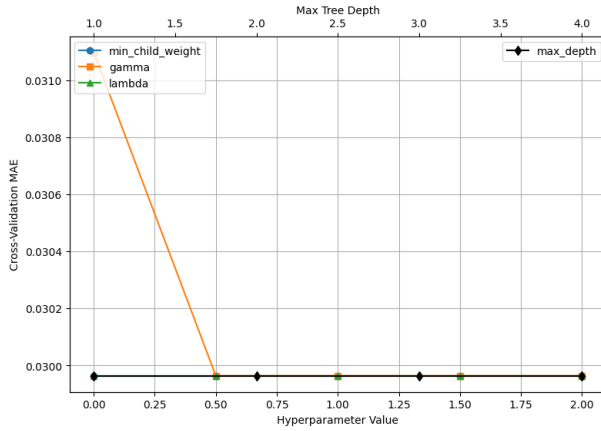
(a) Log return prediction with ARMA



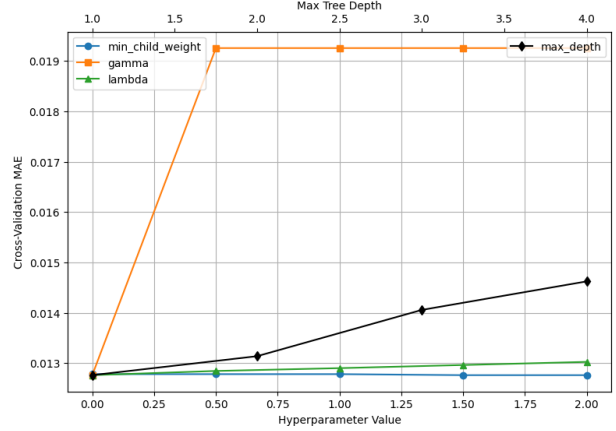
(b) Volatility prediction with GARCH

Figure 3: The cross-validation error against varying numbers of parameters included in the model.

Furthermore, XGBoost did not provide significant improvements in predicting log returns and failed to outperform the benchmark ARIMA model. Given its simplicity and transparency, ARIMA remains the most appropriate choice for return prediction. In contrast, XGBoost clearly surpassed GARCH in predicting the volatility of log returns.



(a) Log return prediction without external features



(b) Volatility prediction without external features

Figure 4: The cross-validation error against XGBoost parameters varying one at a time, plotted on the same coordinate system. Optimal values are used for parameters that are kept fixed.

These results imply that simpler models are preferred for log return prediction, whereas more complex architectures are necessary to achieve higher accuracy in volatility prediction. This aligns with the expectation that log return residuals behave as white noise, containing no further predictive structure, while volatility reflects intricate, non-linear relationships.

Finally, the inclusion of external features did not significantly affect XGBoost's performance in either financial scenario. Figure 1 shows weak correlations between external features and log returns, explaining the lack of improvement. In contrast, Figure 2 indicate strong relationships between external features and volatility, yet model performance remained unchanged across feature subsets. This suggests that while external features may hold predictive value, excessive inclusion risks overfitting, thereby offsetting potential gains.

Conclusion

Predicting the return values of silver and their volatility is of great value for different parties, such as investors, manufacturers, and governments. This assignment set out to examine the effect of model complexity on predictive performance in the financial domain. The benchmark model ARIMA-GARCH was implemented at various levels of complexity to select the optimal configuration that would enhance the performance of predicting log return values of silver in South Africa and their volatility. Similarly, XGBoost models were also trained, selected and assessed with and without appropriate external features to produce comparative results with the benchmark models. This was done to determine the most appropriate model structure, along with the level of complexity for the given financial scenarios and how external features can influence model performance.

After performing model training and selection with 10-fold expanding window cross-validation, it was determined that XGBoost outperforms the ARIMA-GARCH model when predicting volatility but fails to exceed the predictive performance of ARIMA when predicting log returns. The ARIMA (0,0,0) model with a constant mean was selected, which emphasises the absence of meaningful data patterns in the log return time series. Furthermore, although XGBoost models are less interpretable than the benchmark time series models, it provides a level of complexity that can more accurately capture the nonlinear relationships within the data when predicting volatility. This prevents under-fitting and enhances model generalisation.

Including external features in XGBoost models had little effect on the predictive performance, even though some had strong relationships with the target variable. Future studies should further investigate this phenomenon by also implementing Autoregressive Moving Average with Exogenous Inputs (ARMAX) models and tuning the hyperparameters of XGBoost models more carefully. These suggestions should also aim to address the limitations of this study by making timeseries models and XGBoost more comparable and enhancing the hyperparameter tuning of XGBoost.

In conclusion, benchmark models still produce competitive results compared to more complex models, such as XGBoost. Since each model structure provide it's own strengths and weaknesses and has different levels of complexity, it is not sensible to deem one as uniformly superior in all financial scenarios, such as log return prediction and volatility prediction.

Bibliography

- Domingos, P. (2000). A unified bias-variance decomposition. In: *Proceedings of 17th international conference on machine learning*, pp. 231–238.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The elements of statistical learning*. 2nd edn. Springer.
- Viljoen, H. (2025a). Chapter 6: Heteroscedastic time series models. Course notes, Stellenbosch University.
- Viljoen, H. (2025b). Time series analysis honours course notes. Chapters 1–5.